

쓰레기 사진 한 장으로, 재질 판별부터 배출 안내까지

사업과제 : 쓰레기 사진 한 장으로 재질을 판별하고 올바른 분리배출 방법을 안내하는 AI 어시스턴트 구축

 YOLO 게이트

 CNN 전이학습 (EfficientNet-B0)

 VLM 최종 판정

 LangChain RAG

검증 정확도 93.66% · EfficientNet-B0 전이학습, Colab T4 GPU

5일 완주 설계 · 라벨링 불필요 구성 + 단계별 폴백 + 로컬/배포 이중화

담당자 김승현

trash-check.streamlit.app · Streamlit Cloud 배포

목차



1. 프로젝트 개요

추진배경 및 목적 · 과제 범위 · 추진 방법(설계 원칙 · 모델 적용 대상 · 구축 프로세스)



3. 시스템 아키텍처

4단 판별 파이프라인(YOLO → CNN → VLM → RAG) · 단계별 풀백 설계 · 최종 판정 로직 judge()



5. VLM 최종 판정

도입 배경(plastic → can 오분류) · 프롬프트 설계 · 로컬(Ollama)/배포(HF) 이중화



7. 학습 성과 및 결과 시각화

검증 정확도 93.66% · 혼동행렬 · 클래스별 지표 · 한계 분석과 대응



2. 데이터 수집 및 구축

TrashNet 6 → 5클래스 재구성(2,527장) · 클래스 불균형 대응 · RAG 근거 문서 수집



4. CNN 전이학습

EfficientNet-B0 구조 · 데이터 증강 6종 · 하이퍼파라미터 · Colab T4 학습 환경



6. LangChain RAG 챗봇

인덱싱 파이프라인(청크·임베딩·Chroma) · 검색·답변 생성 · 챗봇 UX



8~10. 시연 앱 · 트러블슈팅 · 결론

Streamlit 3탭 구성 · 주요 이슈 5건 · 결과 요약 및 향후 발전 방향

프로젝트 개요

판별(비전 모델)과 안내(LLM + 근거 문서)를 결합한 파이프라인이 필요하다

분리배출은 재질(플라스틱·캔·유리·종이) 판별이 선행되어야 하지만, 일반 시민이 혼합 재질·오염된 쓰레기의 배출 방법을 정확히 알기 어렵다. 이미지 분류 CNN만으로는 "무엇인지"는 알 수 있어도 "어떻게 버려야 하는지"는 답할 수 없다.



목적

사진 업로드 한 번으로 재질 판별 → 분리수거함 안내 → 배출 방법 질의응답까지 이어지는 엔드투엔드 데모 앱 구축



과제 범위

TrashNet 데이터 구축, YOLO 게이트, CNN 전이학습, VLM 최종 판정, LangChain RAG 챗봇, Streamlit Cloud 배포까지 전 과정



설계 원칙

5일 일정 내 완주를 위해 라벨링이 필요 없는 구성을 선택. 각 단계는 실패해도 다음 단계로 안전하게 폴백하며, 로컬(Ollama)·배포(HF API)를 환경변수 하나로 전환하는 이중화 구조를 적용

데이터 수집 및 구축 — TrashNet 재구성

Stanford CS229의 공개 데이터셋 **TrashNet**(Gary Thung & Mindy Yang)을 내려받아, 원본 6클래스를 과제 5클래스로 매핑해 구축했다 (scripts/download_dataset.py). 학습/검증은 8:2로 분할(seed 42 고정)했다.

클래스	장수	원본 TrashNet 클래스	매핑 근거
paper (종이)	997	paper + cardboard	국내 분리배출 기준상 종이류로 통합 배출
glass (유리)	501	glass	그대로 사용
plastic (플라스틱)	482	plastic	그대로 사용
can (캔)	410	metal	생활 쓰레기에서 금속류 대부분이 캔
trash (일반쓰레기)	137	trash	이물질·판별불가 항목의 목적지 클래스
총합	2,527		

⚠ 클래스 불균형 대응 — paper(997)가 trash(137)의 약 7배 → CrossEntropyLoss에 클래스별 가중치 `total / (num_classes × count)` 적용

RAG 근거 문서 수집

지어내지 않고 답하는 챗봇을 위해 공신력 있는 원문 문서를 근거로 사용

PDF 26~33페이지("품목 A-Z 잡화 사전" 부록)는 반복 키워드로 임베딩 유사도를 왜곡시켜 인덱싱에서 제외했다 — 관련 청크가 82개 중 77위로 밀려나는 문제가 확인되어 해당 페이지를 제거하고 재인덱싱했다.



재활용품 분리배출 가이드라인 (PDF)

출처: 환경부(구로구청 게시본) · 용도: 재질별 배출 요령·FAQ·폐기물 분류



재질별 분리배출 요령 (md)

출처: 찾기쉬운 생활법령정보(easylaw.go.kr) · 용도: 법령 근거 포함 재질별 요령

시스템 아키텍처 — 4단 판별 파이프라인

사진 업로드 (쓰레기통 투입구 UI)

[1] YOLO 게이트 model/detect.py COCO 사전학습 YOLOv8n, conf=0.25 (파인튜닝 없음)
0개 감지 → 전체 이미지 사용 / 1개 이상 감지 → 최대 면적 물체 crop

[2] CNN 분류 model/predict.py EfficientNet-B0 전이학습 → 5클래스 확률 (CPU 추론)

[3] VLM 최종 판정 model/vision.py 재질 + 이물질 동시 판단 (로컬 Ollama / 배포 HF API)
사용 가능 → VLM 결과 최종 채택 / 실패 → CNN + 확신도 70% 폴백

[4] RAG 챗봇 rag/chain.py 판별 재질 + 사용자 질문 → Chroma 벡터DB 검색(k=8) → 근거 기반 답변

단계별 폴백 설계

단계	실패 상황	폴백 동작
YOLO 게이트	물체 0개 감지 (COCO 미포함)	전체 이미지를 CNN 입력으로 사용
YOLO 게이트	crop 좌표 계산 예외	원본 이미지로 진행
VLM 판정	토큰 없음·모델 미지원·크레딧 소진	CNN 분류 + 확신도 70% 임계값 판정
RAG 챗봇	LLM 연결 실패·API 장애	safe_ask()가 안내 메시지로 대체

최종 판정 로직 — judge()

```

if vlm_result:
    if vlm_result["contaminated"]:
        return "trash", "이물질 감지 → 세척 후 재활용 가능"
    return vlm_result["material"], "VLM 판정 완료"

if cnn_label == "trash":
    return "trash", "CNN이 일반쓰레기로 분류"
if cnn_conf < 0.70:
    return "trash", "확신도 낮음 → 일반쓰레기로 안내"
return cnn_label, "CNN 판정 완료"

```

⚠ 모든 단계에 안전한 폴백 경로를 두어, 외부 API 장애·크레딧 소진 상황에서도 앱이 항상 동작하도록 설계

CNN 전이학습 (EfficientNet-B0)

입력: (3, 224, 224) — YOLO crop 이미지

|

EfficientNet-B0 백본 (ImageNet 사전학습 가중치, 전체 파인튜닝)

|

classifier[1]: Linear(1280 → 5) # 분류 헤드만 5클래스로 교체

|

출력: 5개 클래스 확률 (plastic / can / glass / paper / trash)

데이터 증강 (6종)

- **RandomResizedCrop** — scale 0.6~1.0, ratio 0.8~1.25 (촬영 거리·구도 변화)
- **RandomHorizontalFlip** — p=0.5
- **RandomRotation** — $\pm 25^\circ$ (기울어진 촬영 각도)
- **ColorJitter** — brightness/contrast/saturation 0.35, hue 0.05
- **RandomPerspective** — distortion 0.2, p=0.3
- **RandomErasing** — p=0.25, scale 0.02~0.15 (오염·가림 시뮬레이션)

하이퍼파라미터 · 학습 환경

- **옵티마이저** AdamW, lr 0.0001, CosineAnnealingLR(T_max=20)
- **배치 크기** 32 · **손실함수** CrossEntropyLoss + 클래스 가중치
- **에폭** 20 (초기 15 → 증강 강화 후 수렴 여유 확보)
- **체크포인트** best val accuracy 시 Drive 저장
- **학습 플랫폼** Google Colab · NVIDIA T4 GPU
- **추론 환경** 로컬 CPU (배포 환경 GPU 없음)

VLM 최종 판정 (재질 + 이물질)

CNN은 TrashNet(흰 배경 스튜디오 사진)으로 학습되어 실제 촬영 사진에서 재질을 혼동하는 사례가 반복됐다 — 대표적으로 기름때 묻은 플라스틱 그릇을 캔으로 오분류. 증강을 강화해 재 학습해도 개선이 뚜렷하지 않아(도메인 차이가 원인), 실사진 일반화 능력이 뛰어난 범용 VLM을 최종 판단자로 승격했다.

프롬프트 설계

이 사진 속 쓰레기 재질을 분류해줘. 다음 중 하나만 골라:
plastic(플라스틱), can(캔/금속), glass(유리),
paper(종이/골판지), trash(이물질이 심해
재활용이 불가능한 상태).
이물질이 묻어 있는지도 판단해서,
반드시 '재질, 이물질여부' 형식으로
한 줄만 답해 (yes 또는 no).
예시: plastic, yes

로컬 / 배포 이중화 (LLM_BACKEND)

환경	백엔드	모델
로컬 데모	Ollama (기본값)	gemma4:e2b (vision)
클라우드 배포	HF Inference API	Llama-3.2-11B-Vision → Qwen2.5-VL-72B → Qwen2-VL-7B 순차 폴백

실패 사유는 조용히 삼키지 않고 raw 필드에 담아 앱의 "VLM 디버그 정보"에서 바로 확인 가능하게 했다. HF 크레딧 소진 등 전면 실패 시에도 CNN 판정으로 폴백된다.

LangChain RAG 챗봇

환경부 가이드라인 PDF + 생활법령 md (잡화 부록 8페이지 제외 → 29문서)

- RecursiveCharacterTextSplitter (chunk 500자, overlap 50자) → 54청크
- 다국어 임베딩 paraphrase-multilingual-MiniLM-L12-v2 (~470MB, Streamlit Cloud 메모리 고려)
- Chroma 벡터DB (rag/chroma_db/) – 배포 첫 부팅 시 자동 구축

🔍 검색 · 답변 생성 (rag/chain.py)

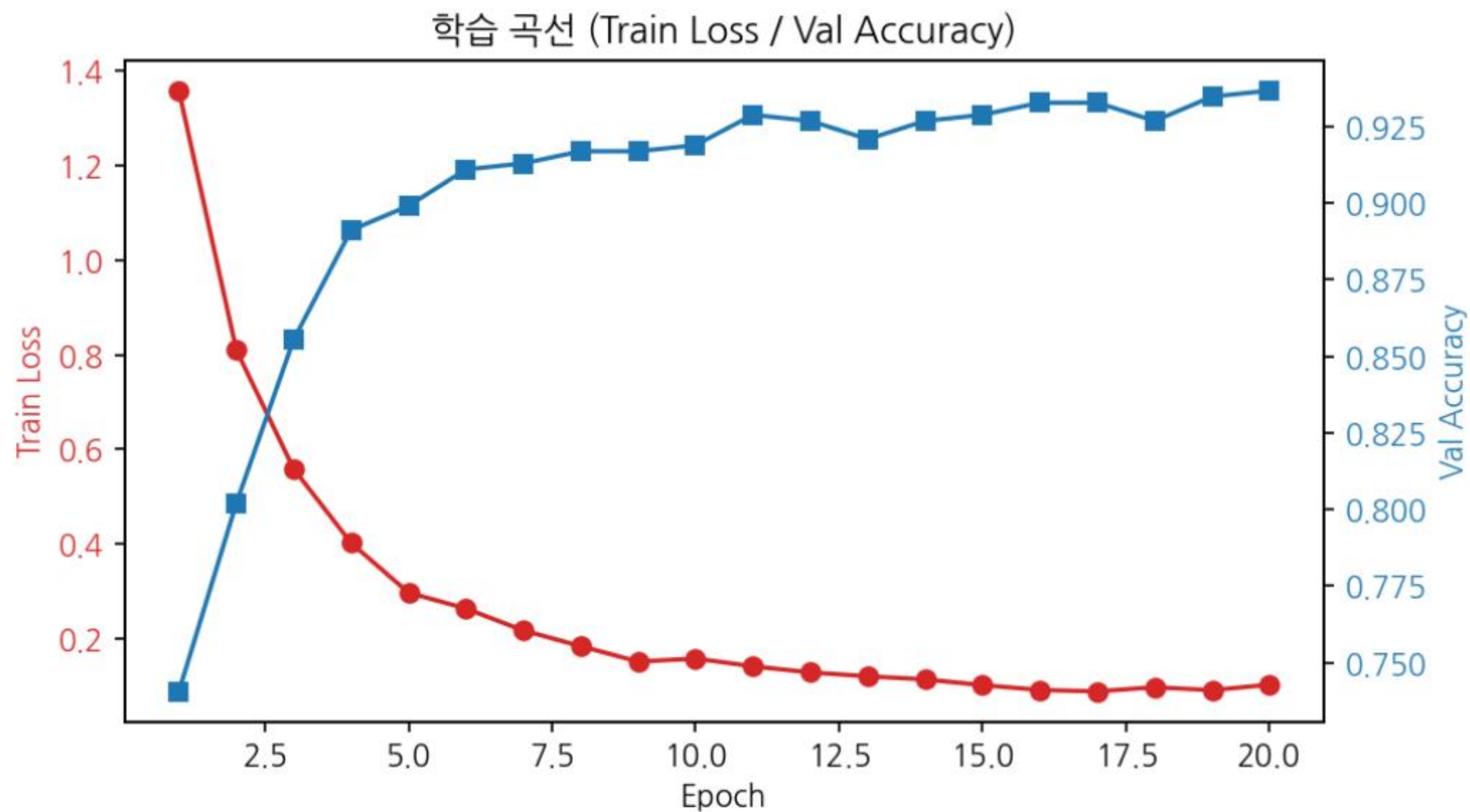
관별된 재질(한국어) + 사용자 질문

- "{재질} 분리배출 방법 {질문}" 쿼리로 벡터 검색 (k=8)
- 프롬프트: 참고 문서 + 감지된 재질 + 질문
"문서에 없는 내용은 지어내지 말고
모른다고 답하세요"
- LLM 답변 생성
(로컬 Ollama qwen2.5:7b /
배포 HF Qwen2.5-7B-Instruct)

💬 챗봇 UX

- 업로드 영역 **우측에 상시 노출**되는 카카오톡 스타일 말풍선 패널
- 관별 완료 시 "이 재질의 기본 분리배출 방법"을 **자동으로 먼저 답변**
- 새 이미지 업로드 시 대화 자동 초기화
- LLM 호출 실패 시 safe_ask()가 안내 메시지로 대체 (앱 다운 방지)

학습 성과 (1) — 학습 곡선



93.66% 최종 검증 정확도

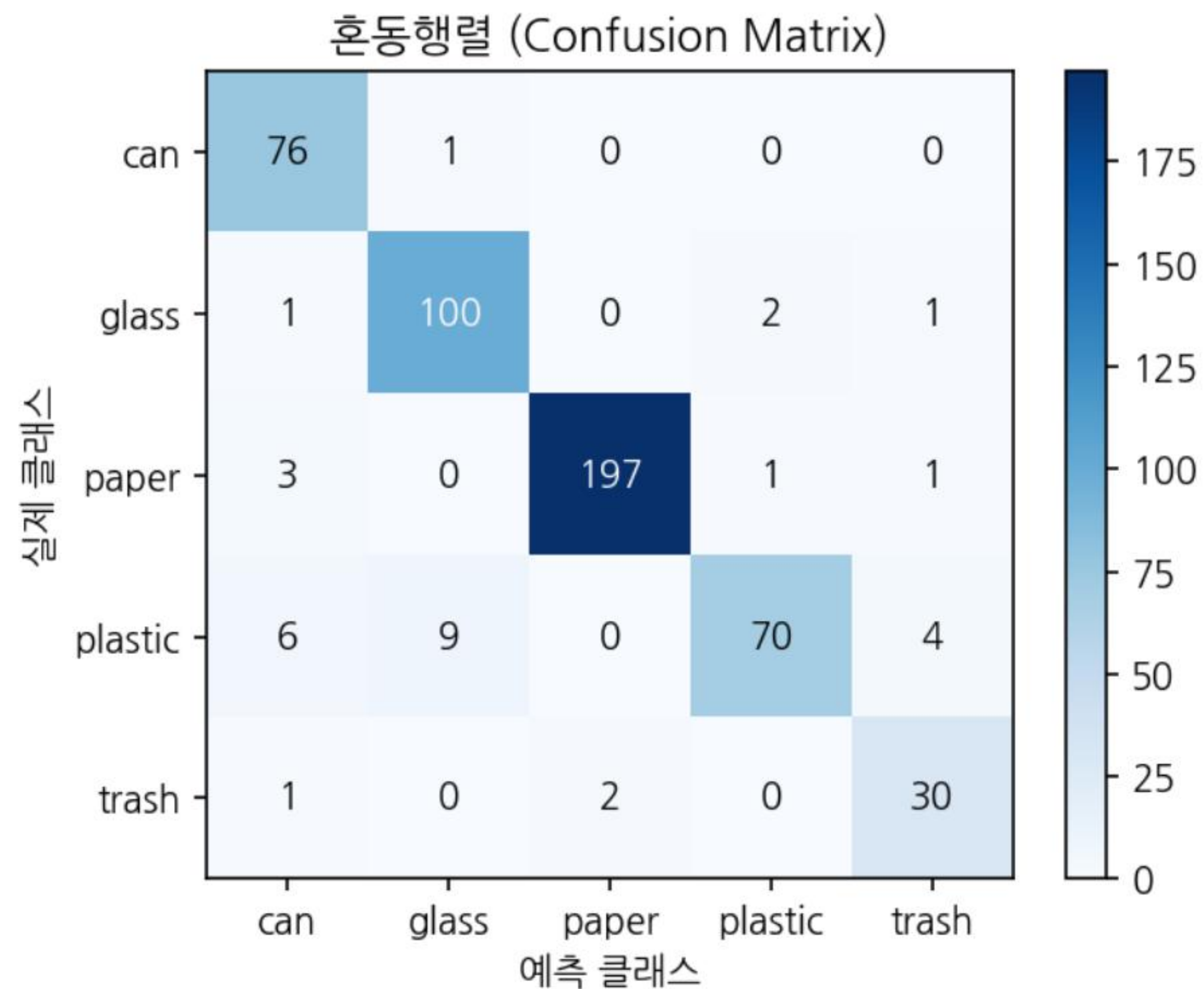
Train Loss

- 1 epoch 1.36 → 20 epoch 약 0.09까지 **꾸준히 감소**
- 급격한 하락 구간(1~4 epoch) 이후 완만하게 수렴
- 과적합 징후 없이 안정적으로 학습 진행

Val Accuracy

- **8 epoch 근처부터 93% 안팎으로 수렴**, 이후 완만한 상승
- CosineAnnealingLR($T_{max}=20$)로 후반 학습률을 점감시켜 미세한 변동 속에서 안정적 유지
- 총 20 epoch 학습 중 best val accuracy 시점을 Drive에 체크포인트 저장

학습 성과 (2) — 혼동행렬



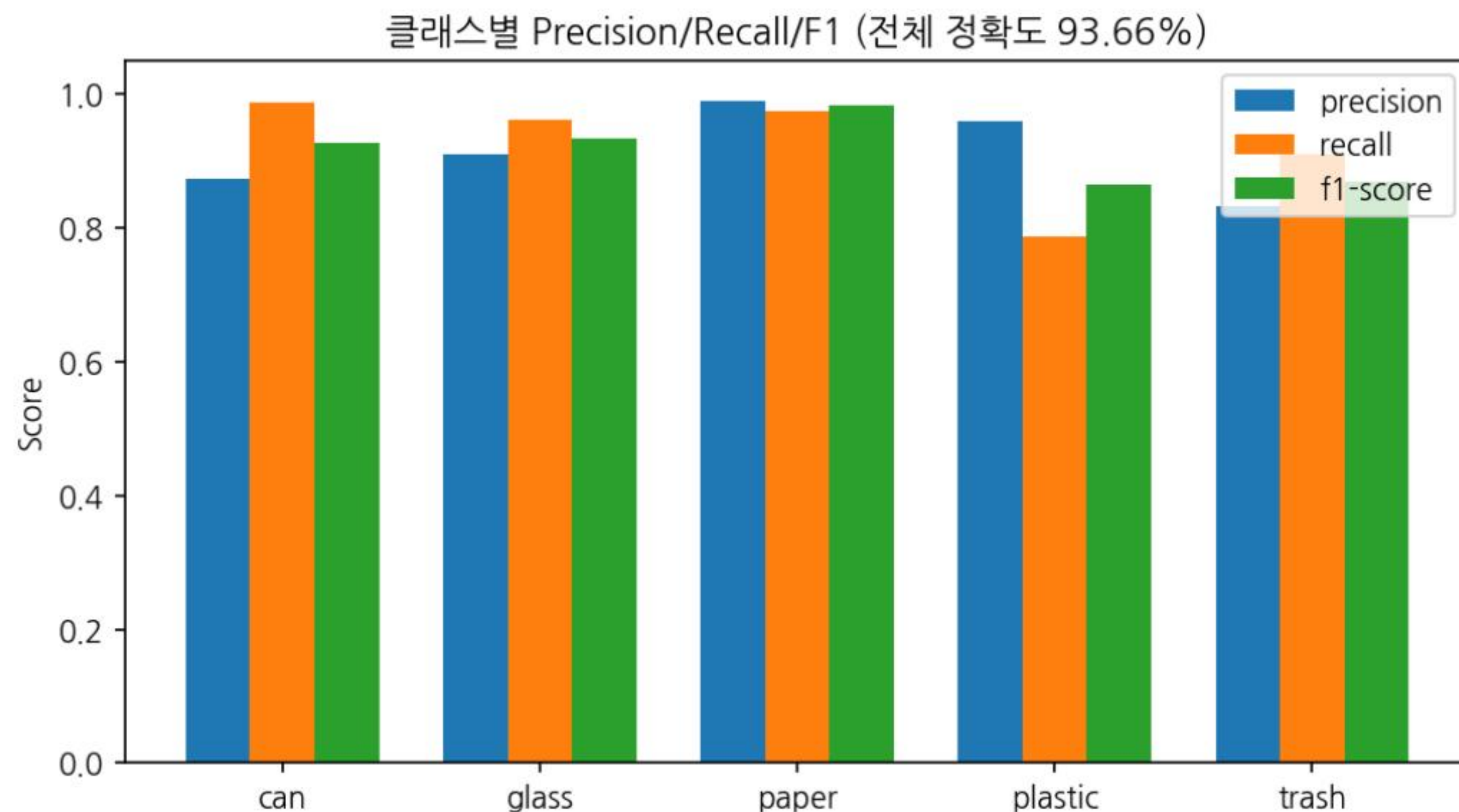
✓ 안정적으로 분류된 클래스

- **can** 76/77건 정확 분류 (오분류 1건 → glass)
- **paper** 197/202건 정확 분류 — 가장 높은 정확도
- **glass** 100/105건 정확 분류
- 세 클래스 모두 대각선에 집중되어 안정적으로 분류됨

⚠ plastic ↔ can 오분류 잔존

- **plastic** 89건 중 70건만 정확 분류 (recall 낮음)
- plastic → glass 9건, plastic → can 6건, plastic → trash 4건으로 오분류
- trash도 paper·can과 일부 혼동(3건) — 소량 클래스(137장)의 한계
- → CNN 단독 판정의 신뢰도 한계를 확인, VLM 최종 판정 도입의 직접적 근거

학습 성과 (3) — 클래스별 Precision/Recall/F1



클래스별 해석

- **paper** precision·recall·F1 모두 90%대 후반 — 가장 안정적
- **can·glass**도 precision·recall 90%대 유지
- **plastic**은 precision은 높지만 **recall이 가장 낮음(약 0.79)** — 혼동행렬의 plastic → glass/can 오분류가 원인
- **trash**는 학습 데이터가 가장 적은 클래스(137장)임에도 클래스 가중치 적용으로 **F1 80%대 확보**

종합 해석 · 대응

- 증강 강화(강한 ColorJitter·Erasing) + 에폭 15 → 20 재학습 → **검증 정확도는 동일 수준**, 증강만으로는 개선 한계
- 원인: TrashNet(흰 배경 스튜디오)과 실제 촬영 사진의 **도메인 차이**
- 대응: 검증셋 성능을 더 쥐어짜는 대신 **VLM을 최종 판단자로 승격**하는 아키텍처 차원의 해결 선택 (5절)

Streamlit 시연 앱 (1) — 판별 데모 · 초기 화면



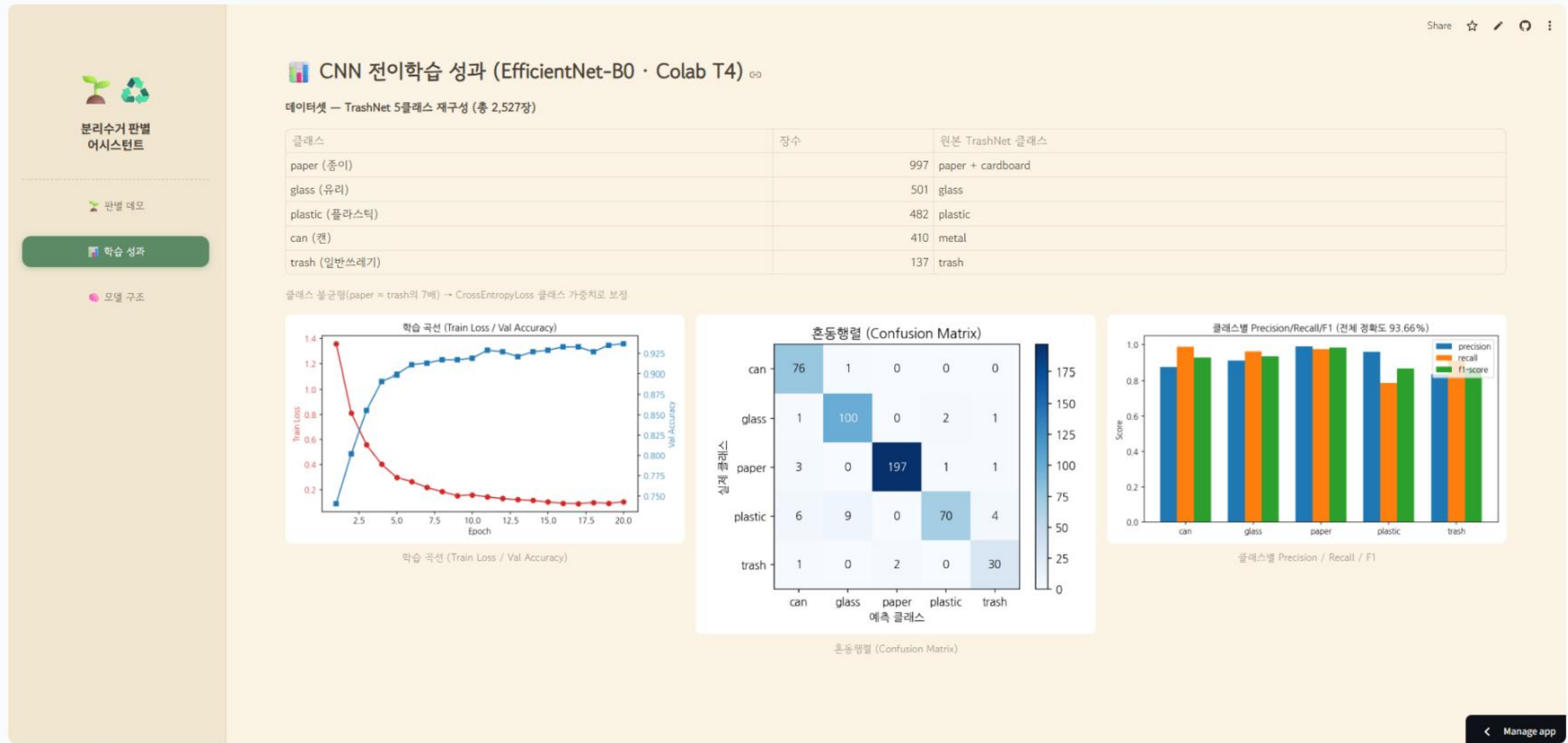
초기 화면 — st.file_uploader를 쓰레기통 모양으로 리스타일해 "던져 넣는" 은유 구현 — 업로드 전에는 플라스틱·캔·유리·종이·일반쓰레기 5종 수거함과 안내 문구만 노출

Streamlit 시연 앱 (2) — 판별 데모 · 결과 화면



판별 완료 화면 — 파일 업로드 후 판정 결과 배지에 "재질 판별 완료! (VLM 판정)"과 함께 세척·라벨 제거·압착 등 구체적인 배출 방법을 자동으로 안내, 우측 챗봇 패널에서 추가 질의응답 가능

Streamlit 시연 앱 (3) — 학습 성과 탭



Streamlit 시연 앱 (4) — 모델 구조 탭



모델 구조 탭 실제 화면 — 4단 파이프라인 다이어그램과 YOLO 파인튜닝 없음 · VLM 최종 판단자 승격 등 5가지 설계 결정과 그 이유를 함께 제공

▶ 실행 방법

```
# 로컬 (Ollama)
ollama pull qwen2.5:7b    # 챗봇 LLM
ollama pull gemma4:e2b    # VLM 판정
.venv\Scripts\streamlit run app.py    # 배포: LLM_BACKEND="hf", HF_TOKEN 시크릿 설정
```


트러블슈팅 — 주요 이슈 5건

#	이슈	원인 → 해결
1	Streamlit Cloud cv2 import 실패	GUI용 opencv가 서버에서 libGL.so.1 부재로 실패 → opencv-python-headless + packages.txt(libgl1)
2	YOLO 게이트 오판	COCO 80클래스에 캔·종이가 없어 0개/여러 개 오판 → 0개는 전체 이미지 폴백, 여러 개는 최대 박스 자동 선택
3	HF Provider 제약·크레딧 소진	계정별 지원 모델 상이(model_not_supported)·402 오류 → 다중 모델 순차 폴백 + 실패 사유 화면 노출 + CNN 폴백
4	RAG가 무관한 문서만 반환	PDF 잡화 부록의 반복 키워드가 임베딩 유사도 왜곡(관련 청크가 82개 중 77위) → 해당 페이지 인덱싱 제외
5	로컬에서 VLM이 항상 실패	VLM이 HF API 전용 구현 → LLM_BACKEND 스위치로 로컬 Ollama 이중화

 상세 경위는 TROUBLESHOOTING.md 참고

결론 및 고찰

5일 일정 내에 **사진 업로드** → **재질 판별** → **분리수거함 안내** → **배출 방법 질의응답**까지 이어지는 엔드투엔드 파이프라인을 완성하고 Streamlit Cloud에 배포했다. CNN 전이학습으로 검증 정확도 **93.66%**를 달성했고(클래스 불균형 7배는 가중치로 보정), 실사진 한계(plastic ↔ can 혼동)는 데이터가 아닌 **아키텍처(VLM 최종 판단자 승격)**로 해결했다. 모든 단계에 폴백을 설계해 외부 API 장애·크레딧 소진 상황에서도 앱이 항상 동작하며, 로컬/배포를 환경변수 하나로 전환하는 이중화로 데모·배포 환경을 모두 지원한다.

📁 구현 산출물

- `scripts/download_dataset.py` — TrashNet 다운로드 + 6 → 5클래스 재구성
- `model/detect.py` · `predict.py` · `vision.py` — YOLO 게이트 · CNN 추론 · VLM 판정
- `rag/ingest.py` · `chain.py` — 문서 인덱싱, 검색(k=8) + LLM 답변
- `app.py` — 쓰레기통 UX + 챗봇 패널 (3탭) → Streamlit Cloud 배포

🚀 향후 발전 방향

- **실사진 파인튜닝** — CNN 재학습으로 도메인 차이 근본 해소, VLM 의존도 감소
- **YOLO 쓰레기 특화 파인튜닝** — 다중 물체 동시 판별로 확장
- **RAG 하이브리드 검색** — BM25(키워드)+벡터 결합으로 재현율 향상
- **지자체별 배출 기준 반영 · 판별 이력 대시보드** — 지역 맞춤화 및 오분류 패턴 분석